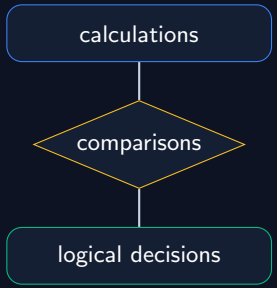
Python Foundations

From mathematical operations to logical decisions

Learn to use operators, compare values, and build algorithms using conditions

Week 5



Mathematical Operations in Python

CALCULATIONS

The Python language supports basic arithmetic operations, as well as a few special ones (integer division, exponentiation).

Python provides operators for addition, subtraction, multiplication, division, and other common calculations.

Remember:

The / operator performs "floating-point" division, while // rounds the result down.

```
a = 15
b = 4

print(a + b)   # 19 (Addition)
print(a - b)   # 11 (Subtraction)
print(a * b)   # 60 (Multiplication)
print(a / b)   # 3.75 (Division)

print(a // b) # 3 (floor quotient)
print(a % b)   # 3 (remainder of
    division)
print(a ** b) # 50625 (exponentiation)
```

Relational Operators

COMPARISONS

Relational operators compare two values and produce a logical result: `True` or `False`.

Pseudocode vs. Python

Equal to: = → ==

Not equal to: ≠ → !=

Greater/less than: > , <

Greater than or equal: ≥ → >=

Less than or equal: ≤ → <=

Attention: = assigns a value, while == checks for equality.

```
print(5 == 5)   # True
print(5 != 3)   # True
print(7 > 4)    # True
print(2 < 6)    # True
print(8 >= 8)   # True
print(3 <= 2)   # False
```

Combining Conditions

LOGIC

Logical operators allow us to combine multiple conditions. In Python we use `and`, `or`, and `not`.

- **AND:** both conditions must be true for the result to be True.
- **OR:** it is enough for at least one condition to be true.

```
grade = 8
has_id = True

# Example with AND
if grade > 5 and has_id:
    print("You can enter the exam.")
else:
    print("You cannot enter the exam.
        ")

# Example with OR
is_saturday = True
is_sunday = False

if is_saturday or is_sunday:
    print("It is the weekend!")
```

The if Statement

DECISIONS – Simple alternative structure

The `if` statement executes a block of code only if a condition is true. If it is false, the block is skipped.

Important rule:
The condition ends with a colon (`:`), and the code block must be indented.

```
# Pseudocode:
# IF grade >= 5 THEN
#     print "You passed the exam!"
# END IF

# Python:
grade = 8
if grade >= 5:
    print("You passed the exam!")
```

The if...else Statement

TWO BRANCHES – Complete alternative structure

The `if...else` structure provides two branches: one for when the condition is true, and another for when it is false.

Key idea:

If the remainder of an integer divided by 2 is 0, the number is even; otherwise, it is odd.

```python
# Python:
number = 7
if number % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

The if...elif...else Statement

MULTIPLE BRANCHES – Multiple alternative structure

We use `elif` to check multiple conditions, in order. The block of the first true condition is executed; the other branches are ignored.

Remember:

The order of conditions matters. Once a condition is true, Python stops checking the following branches.

```
temp = 15

if temp > 30:
    print("Go to the pool.")
elif temp >= 20:
    print("Perfect weather.")
elif temp >= 10:
    print("Take a jacket.")
else:
    print("Stay indoors.")
```

Exercises: Calculations and Decisions

TEST YOUR KNOWLEDGE

1. Discounted ticket

Write a program that checks if a person qualifies for a discounted cinema ticket. The discount applies if the age is **under 18** OR **over 65** (inclusive).

Hint: Use the `or` operator and the `if...else` structure.

2. Lucky number

A number is considered "lucky" if it is strictly **positive** AND **even**. Write a code that checks a variable `n` and prints "Lucky" or "Ordinary".

Hint: Use the `and` operator and check the remainder of the division (`% 2 == 0`).

Integration Exercise

MINI-PROJECT

Grade Classification

You have the variable `score` (from 0 to 100). Using the `if...elif...else` structure, print the corresponding grade:

- ≥ 90: Excellent
- ≥ 70 and < 90: Good
- ≥ 50 and < 70: Sufficient
- < 50: Insufficient

Remember: The order of conditions in `elif` helps you avoid checking both limits (e.g., if it failed the ≥ 90 test, you already know it is < 90).

```
score = 75

# Write your code below:
if score >= 90:
    print("Excellent")
elif score >= 70:
    print("Good")
# Continue with the rest...
```

Set I: Algebraic Evaluations

PRACTICAL EXERCISES

Problem 1: Parity and Sign

Write a program that takes an integer `n` and prints the message "Valid" if it is simultaneously strictly positive and even, or "Invalid" otherwise.

Hint: Use the logical operator `and` and check the remainder of the division (`n % 2 == 0`).

Problem 2: Membership in Unions

Develop an algorithm that checks if a real number `x` belongs to the union of disjoint intervals $I = (-\infty, -10] \cup [10, \infty)$, printing the message "Belongs" if affirmative, respectively "Does not belong" otherwise.

Hint: Use the `or` operator to join the two inequalities.

Set II: Inequalities and Divisibility

PRACTICAL EXERCISES

Problem 3: Triangle Inequality

Given three strictly positive real numbers a, b, and c, design a decision structure that prints "Valid triangle" if the sum of any two sides is strictly greater than the third, or "Invalid" otherwise.

Hint: Connect three relational expressions with the logical operator `and`.

Problem 4: Compound Divisibility

Write a code that evaluates a natural number `N` and returns the text "Multiple of 3 and 5" if it is simultaneously divisible by both numbers, otherwise "Condition not met".

Hint: Evaluate simultaneously the expressions `N % 3 == 0` and `N % 5 == 0`.

Integration Project: Piecewise Functions

MULTIPLE DECISION STRUCTURES

Problem 5: Classification of Continuous Values

Implement a piecewise function that takes as input a real score p $\in [0, 100]$ and prints a corresponding classification: "Excellent" if $p \in [90, 100]$, "Good" if $p \in [70, 90)$, "Sufficient" if $p \in [50, 70)$, respectively "Insufficient" if $p \in [0, 50)$.

Remember: The order of the `if...elif` structure allows omitting the evaluation of the upper limit for the lower intervals.

```
p = 75.5

# Piecewise function
    implementation:
if p >= 90:
    print("Excellent")
elif p >= 70:
    print("Good")
# Complete the structure up
    to
# the 'else' branch included.
```